be testable. For example, an instant messaging API might have the functional requirement that *users are able to read their messages*. To preserve confidentiality, you may then add constraints that users are only able to read their *own* messages, and that a user must be *logged in* before they can read their messages. In this approach, security goals become constraints on existing functional requirements. It then becomes much easier to think up test cases. For example:

- Create two users and populate their accounts with dummy messages
- Check that the first user cannot read the messages of the second user
- Check that a user that has not logged in cannot read any messages

There is no single correct way to break down a security goal into specific requirements, and so the process is always one of iteration and refinement as the constraints become clearer over time, as shown in figure 1.5. After identifying assets and defining security goals, you break down those goals into testable constraints. Then as you implement and test those constraints, you may identify new assets to be protected. For example, after implementing your login system you may give each user a unique temporary session cookie. This session cookie is itself a new asset that should be protected.